

DocFlow collaborative editor

A concise, reproducible record of automated, integration, browser, and container verification for the completed assessment application.

Candidate	Muhammad Owais Nizami	Verified	3 September 2026 (PKT)
Repository	github.com/mowaisnizami/ajaia-assessment	Runtime	Docker Desktop / Linux containers

Overall result

PASS - 10/10 gates

The application builds, starts as a three-service stack, persists rich text in PostgreSQL, imports files, shares documents, and rejects viewer edits at the API.

Angular unit tests and production build

PASS

2 tests passed; optimized production bundle generated in a clean Node 22 Alpine build stage.

NestJS unit tests and production build

PASS

2 suites and 3 tests passed; NestJS TypeScript compilation completed successfully.

Real PostgreSQL integration workflow

PASS

Create, update, reopen, import, share, list, and viewer-denial assertions passed over HTTP.

Browser acceptance verification

PASS

Owner controls rendered; collaborator viewer mode disabled the title and Save button; rich text reopened.

Container deployment

PASS

PostgreSQL and API reported healthy; Nginx web application served at localhost:8080.

Coverage boundary

This evidence validates the assessment build and its documented mocked-identity model. It does not claim production OIDC, migrations, load testing, or real-time co-editing; those items are explicitly recorded as production follow-ups.

Automated and integration results

Gate	Evidence	Result
NestJS unit	Health controller and access-policy service	PASS
NestJS build	nest build; no TypeScript errors	PASS
Angular unit	app.spec.ts: 1 file, 2 tests	PASS
Angular build	Production bundle: 339.91 kB initial / 87.46 kB estimated transfer	PASS
Health	GET /api/health returned status=ok	PASS
Persistence	Rich-text HTML was identical after save and reopen	PASS
Import	Markdown upload created persistent document import-sample	PASS
Sharing	Collaborator list and reopen returned access=VIEWER	PASS
Authorization	Viewer PATCH was rejected with HTTP 403	PASS
Browser UI	Owner editable; viewer title and Save disabled	PASS

Live HTTP integration output

```
{
  "health": "ok",
  "usersSeeded": 2,
  "createdTitle": "Assessment launch plan",
  "richTextPersisted": true,
  "importedTitle": "import-sample",
  "sharedDocumentListed": true,
  "reopenedAccess": "VIEWER",
  "viewerEditStatus": 403
}
```

Authorization assertions

Operation	Owner	Editor	Viewer	Unshared
Read / reopen	Allowed	Allowed	Allowed	Denied
Rename / edit	Allowed	Allowed	403	Denied
Grant access	Allowed	Denied	Denied	Denied

Containers, CI, and reproduction

The same Linux-compatible multi-stage Dockerfiles are used locally and by GitHub Actions. Tests execute before runtime image creation, making a passing image a delivery gate.

Service	Image / state	Published port
web	ajaia-assessment-web:latest - running	8080 -> 80
api	ajaia-assessment-api:latest - healthy	3000 -> 3000
db	postgres:17-alpine - healthy	5432 -> 5432

API IMAGE

144.9 MB

NestJS runtime + production dependencies

WEB IMAGE

21.1 MB

Nginx + optimized Angular assets

GitHub Actions behavior

- Runs for every branch push and for pull requests targeting main.
- Build matrix independently verifies API and web Dockerfiles.
- Unit tests and production compilation run inside the Docker build stages.
- Pushes publish SHA-tagged images to GHCR; the default branch also receives latest.
- GitHub Actions cache scopes are isolated per image.

Reproduce locally

```
docker compose up --build -d
docker compose ps
curl --fail http://localhost:3000/api/health
node server/test/http-integration.mjs
# Open http://localhost:8080
```

Reviewer conclusion

The evidence supports a complete assessment-grade product slice: the UI, API, database, authorization boundary, Docker delivery, and commit-triggered image pipeline are working. The only remaining delivery actions are external-account tasks: push the prepared commit, deploy to the candidate's public server, and record the walkthrough video.

READY FOR REVIEW